Table des matières

Cours 08 — Lire des séries temporelles 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Une courbe, lue comme un historien · Sources 2
Étape 2 — Qualifier la série · Sources 3
Étape 3 — Écrire `sql/argent_torrefaction.sql` · RAW 4
Étape 4 — L'argent à mi-mois, puis au mois · RAW 6
Étape 5 — Contrôler l'argent contre le journal · RAW 7
Étape 6 — Versionner · RAW 8
À retenir 8

Cours 08 — Lire des séries temporelles

Cours 8/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
✓ ici		

Ce que vous allez faire

Une courbe de torréfaction est une série temporelle : une mesure prise par un appareil, à intervalle fixe, que personne ne relit. Avant de s'en servir, on la qualifie : sa clé, le type de son temps, son pas, ses sondes, son horloge. Boréal le fait au même cours sur l'historien d'une usine, irrégulier et plein de pièges ; le torréfacteur de La Brûlerie est propre, et ce cours le prouve, requête par requête. Puis vous écrivez `sql/argent_torrefaction.sql`, qui ramène chaque lot à une ligne de l'argent, et vous contrôlez cette ligne contre ce que le torréfacteur a déclaré.

les courbes de la `RAW` → une courbe lue en instants → la série qualifiée, requête par requête → `sql/argent_torrefaction.sql`, une ligne par lot → l'argent à mi-mois, puis au mois → l'argent contrôlé contre le journal des lots

Le code complet est dans `Codes_08_DE.md`, à ouvrir après avoir essayé.

© 2026 iAlexMG. Tous droits réservés.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, DuckDB en ligne de commande et Git ; la seconde pour le notebook, qui s'ouvre à l'étape 1. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
dir /b
REM Attendu : 01_de, plateforme et sources, plus ce qu'Analytics Engineering y a laissé.
```

Ce cours part de l'entrepôt chargé jusqu'au 30 juin, tel que le cours 06 le laisse. S'il manque, reprenez la fin du cours 06.

Étape 1 — Une courbe, lue comme un historien · Sources

Un historien d'usine range chaque mesure sous un instant absolu. Le torréfacteur, lui, compte en secondes depuis la charge du lot : l'instant d'une lecture est le début du lot, que donne le journal, plus sa seconde. `INTERVAL 1 SECOND` est une durée d'une seconde ; multipliée par un nombre, elle en donne autant.

1.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 08 ». Écrire les deux premières cellules, comme au cours 03 : la première attache l'entrepôt en lecture seule, l'exécuter ; la seconde le détache, l'écrire sans l'exécuter.

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/brulerie.duckdb' AS entrepot (READ_ONLY);
-- Attendu : entrepot paraît sous Attached databases.
```

```
DETACH DATABASE IF EXISTS entrepot;
```

1.2 Lire les moments marqués du lot L-2025-0214, chacun à son instant, dans une cellule neuve.

```
-- 1. les lectures marquées d'un lot, chacune à son instant : le début du lot, plus sa seconde
SELECT c.seconde, CAST(l.debut AS TIMESTAMP) + CAST(c.seconde AS INTEGER) * ____ AS instant,
       c.temp_grain, c.temp_air, c.evenement
FROM entrepot.raw_courbes AS c
JOIN entrepot.raw_lots AS l USING (lot_id)
WHERE c.lot_id = 'L-2025-0214' AND c.evenement IS NOT NULL
ORDER BY CAST(c.seconde AS INTEGER);
-- Attendu : 4 lignes : CHARGE à 07:00:00, POINT_BAS à 07:01:40, PREMIER_CRAQUEMENT à 07:11:00,
-- SORTIE à 07:13:00.
```


© 2026 iAlexMG. Tous droits réservés.

Deux sondes, lues ensemble toutes les 10 secondes : le grain et l'air du tambour. À la charge, la sonde du grain baigne encore dans le tambour chaud, 200,2 °C ; le grain froid la fait tomber à 87,6 °C au point bas, puis la chauffe remonte jusqu'au premier craquement et à la sortie.

Étape 2 — Qualifier la série · Sources

Une série se qualifie avant de servir, et chaque question se règle par une requête :

La question	Ce qu'on vérifie
la clé	un seul point par lot et par seconde
le type du temps	ce que le moteur fait de la colonne `seconde`
le pas	10 secondes entre deux lectures d'un même lot, partout
les sondes	aucune lecture vide
l'horloge	aucun changement d'heure pendant le mois
les moments marqués	chacun une fois par lot

2.1 La clé, dans une cellule neuve.

```sql
-- 1. les couples (lot, seconde) qui reviennent plus d'une fois
SELECT lot_id, seconde, COUNT(*) AS fois
FROM entrepot.raw_courbes
GROUP BY lot_id, seconde
HAVING ____;
-- Attendu : aucune ligne.
```

2.2 Le type du temps. La `RAW` garde tout en texte, et un texte se trie caractère par caractère.

```sql
SELECT seconde FROM entrepot.raw_courbes WHERE lot_id = 'L-2025-0214' ORDER BY seconde LIMIT 5;
-- Attendu : 0, 10, 100, 110, 120.
```

En texte, 100 vient avant 20 : toute requête qui met le temps en ordre le convertit d'abord.

2.3 Le pas. `LAG(x) OVER (PARTITION BY … ORDER BY …)` rend, sur chaque ligne, la valeur de `x` à la ligne précédente de sa partition ; la première ligne n'en a pas, et reçoit `NULL`. `LAG` s'apprend au Cours 04 de la formation SQL avancé, `Theorie_04_SQL.md`, section 4.

```sql
-- 1. pour chaque lecture, l'écart avec la lecture précédente du même lot, dans l'ordre du temps
WITH pas AS (
    SELECT CAST(seconde AS INTEGER) - LAG(CAST(seconde AS INTEGER))
           OVER (PARTITION BY ____ ORDER BY CAST(seconde AS INTEGER)) AS ecart
```


© 2026 iAlexMG. Tous droits réservés.

```
    FROM entrepot.raw_courbes
)
-- 2. combien de lectures pour chaque écart
SELECT ecart, COUNT(*) AS lectures
FROM pas
GROUP BY ecart
ORDER BY ecart;
-- Attendu : 2 lignes : 10 pour 3671 lectures, et NULL pour 47, la première lecture de chaque lot.
```

Sans la conversion dans l'`ORDER BY`, la même requête rend 31 écarts différents, dont des écarts négatifs : le piège de 2.2, en grand. Un pas fixe simplifie tout ce qui suit, parce que chaque lecture vaut 10 secondes : une moyenne simple y égale la moyenne pondérée par le temps. Boréal n'a pas cette chance : son historien ne garde un point que quand la valeur bouge assez, et chaque point s'y pondère par la durée pendant laquelle il valait.

2.4 Les sondes. Une sonde qui ne répond pas laisse sa colonne vide, et `COUNT` d'une colonne ne compte que ses valeurs présentes.

```
-- 1. les lectures, puis celles où chaque sonde a répondu
SELECT COUNT(*) AS lectures, COUNT(____) AS grain, COUNT(temp_air) AS air
FROM entrepot.raw_courbes;
-- Attendu : 3718, 3718, 3718.
```

2.5 L'horloge. Le journal écrit le début de chaque lot en heure locale, sans fuseau, comme l'historien de Boréal.

```
SELECT MIN(CAST(debut AS TIMESTAMP)) AS premier, MAX(CAST(debut AS TIMESTAMP)) AS dernier
FROM entrepot.raw_lots;
-- Attendu : 2025-06-03 07:00:00 et 2025-06-27 07:55:20.
```

Deux fois par an, une heure locale se répète ou n'existe pas : en 2025, le 9 mars et le 2 novembre. Les lots de juin n'en croisent aucun, et une courbe compte en secondes depuis sa charge : aucune heure ne s'y répète, aucune n'y manque.

2.6 Les moments marqués.

```
SELECT evenement, COUNT(*) AS lectures FROM entrepot.raw_courbes GROUP BY evenement ORDER BY evenement;
-- Attendu : 5 lignes : CHARGE, POINT_BAS, PREMIER_CRAQUEMENT et SORTIE à 47 chacun, puis NULL à 3530.
```

Le mois est propre : une clé sans doublon, un pas de 10 secondes partout, aucune sonde muette, aucun changement d'heure, quatre moments par lot. Rien à nettoyer, et c'est prouvé : l'argent peut se tirer des points tels quels.

Étape 3 — Écrire `sql/argent_torrefaction.sql` · RAW

L'argent est la couche qui suit le bronze : des données typées, contrôlées, au grain où on les consomme. Ici, une ligne par lot, tirée de ses lectures. Trois décisions font le script :

© 2026 iAlexMG. Tous droits réservés.

- chaque mesure se tire des points : la seconde du premier craquement, celle de la sortie, la température du grain à la sortie ; le journal du torréfacteur ne sert qu'à les contrôler ;
- seuls les lots dont la courbe est arrivée y entrent : le journal connaît le mois d'avance (le second constat du cours 01), et un lot sans courbe n'a rien à dire. La jointure interne, `JOIN`, fait ce tri d'elle-même ;
- l'argent se refait en entier à chaque passage, comme le bronze : 47 lots se recalculent en un instant.

Pour remplir	**L'outil**
une mesure prise à un moment marqué	`MAX(…) FILTER (WHERE evenement = '…')` : l'agrégat ne voit que les lectures qui passent le filtre ; un seul moment par lot, donc le maximum est sa valeur
la sortie, le dernier moment	`SORTIE`, le nom que le dictionnaire donne à l'événement
le développement	la part de la torréfaction qui suit le premier craquement : la durée moins le premier craquement, divisée par la durée, fois 100
une table refaite à chaque passage	`CREATE OR REPLACE TABLE … AS`, vu au cours 07

L'agrégat filtré, `FILTER`, s'apprend au Cours 06 de la formation SQL avancé, `Theorie_06_SQL.md`, section 6. Une division rend un nombre à virgule flottante, même entre deux entiers : le développement se convertit en `DECIMAL(4, 1)`, qui l'arrondit au dixième. Le script tourne dans l'entrepôt : il nomme ses tables sans `entrepot.`.

3.1 Créer le fichier, dans la première fenêtre.

```
notepad 01_de\sql\argent_torrefaction.sql
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous `____` compris :

```
-- 01_de/sql/argent_torrefaction.sql : une ligne par lot, tirée de ses lectures aux 10 secondes
CREATE OR REPLACE TABLE argent_lots AS
WITH courbes AS (
    -- 1. par lot, ce que les sondes ont vu : les lectures, le premier craquement, la sortie
    SELECT lot_id, COUNT(*) AS lectures,
           MAX(CAST(seconde AS INTEGER)) ____ (WHERE evenement = 'PREMIER_CRAQUEMENT') AS craquement_s,
           MAX(CAST(seconde AS INTEGER)) FILTER (WHERE evenement = '____') AS duree_s,
           ____ AS temp_sortie
    FROM raw_courbes GROUP BY lot_id
)
-- 2. le journal du torréfacteur, pour les seuls lots dont la courbe est arrivée
SELECT l.lot_id, CAST(l.debut AS TIMESTAMP) AS debut, l.profil, c.lectures, c.craquement_s,
```


© 2026 iAlexMG. Tous droits réservés.

```
       c.duree_s, c.temp_sortie,
       -- 3. le développement : la part de la torréfaction qui suit le premier craquement, en %
       CAST(____ AS DECIMAL(4, 1)) AS developpement_pct
FROM raw_lots AS l
JOIN courbes AS c USING (lot_id);

-- 4. les lots de l'argent, contre ceux que le journal connaît déjà
SELECT (SELECT COUNT(*) FROM argent_lots) AS argent, (SELECT COUNT(*) FROM raw_lots) AS journal;
```

Enregistrer, puis fermer le Bloc-notes. Comme le bronze, le script finit par son propre contrôle.

Étape 4 — L'argent à mi-mois, puis au mois · RAW

Le journal des lots arrive en entier dès le premier jour ; les courbes, jour après jour. Pour voir l'argent faire le tri, on le lance sur une `RAW` chargée jusqu'au 11 juin. La `RAW` se refait des sources par le pipeline : on l'efface sans rien perdre, par `vider_tables_de.py`, comme au cours 06, qui laisse en place ce qui n'est pas à Data Engineering. `duckdb plateforme\entrepot\brulerie.duckdb -f <fichier>` ouvre l'entrepôt en écriture et y exécute le fichier : chaque passage se fait donc en trois temps, comme un chargement.

4.1 Dans le notebook, relancer la seconde cellule, le `DETACH`. Puis, dans la première fenêtre :

```
python ..\01_Data_Engineering\01_Brulerie_facile\Paquet_de_depart\vider_tables_de.py
REM Attendu : Tables de Data Engineering effacées : 9, ou 10 si argent_lots existe déjà.
python 01_de\pipeline.py 2025-06-11
REM Attendu : bronze : 1527 tickets rangés, puis raw : 30 fichiers au journal.
duckdb plateforme\entrepot\brulerie.duckdb -f 01_de\sql\argent_torrefaction.sql
REM Attendu : argent 18, journal 47.
```

Si la commande s'arrête sur	La cause, et le geste
`brulerie.duckdb est ouvert ailleurs, sans doute par le notebook`, ou `IO Error: Cannot open file "…\brulerie.duckdb", puis File is already open in …\duckdb.exe`	le notebook tient l'entrepôt : relancer le `DETACH`, puis la commande
`Binder Error: Referenced column "…" not found in FROM clause!`	un nom de colonne mal écrit ; DuckDB propose les noms proches sous `Candidate bindings`, et le dictionnaire, section 4, les donne tous
`Conversion Error: Could not convert string 'CHARGE' to INT32`	la conversion en `INTEGER` porte sur `evenement` au lieu de `seconde`

4.2 Relancer la première cellule du notebook, l'`ATTACH`, puis compter les lots de l'argent par jour.

```
SELECT CAST(debut AS DATE) AS jour, COUNT(*) AS lots
FROM entrepot.argent_lots
GROUP BY jour
```


© 2026 iAlexMG. Tous droits réservés.

```
ORDER BY jour;
-- Attendu : 3 lignes : 2025-06-03 à 7 lots, 2025-06-06 à 2, 2025-06-10 à 9.
```

18 lots sur 47 : ceux des trois jours dont la courbe est arrivée au 11 juin. Les 29 autres sont au journal, mais l'argent n'en dit rien tant que leurs sondes n'ont pas parlé.

4.3 Charger le mois en trois temps, puis refaire l'argent, deux fois. Dans le notebook, le `DETACH` ; puis, dans la première fenêtre :

```
python 01_de\pipeline.py 2025-06-30
REM Attendu : bronze : 1527 tickets rangés, puis raw : 75 fichiers au journal.
duckdb plateforme\entrepot\brulerie.duckdb -f 01_de\sql\argent_torrefaction.sql
REM Attendu : argent 47, journal 47.
duckdb plateforme\entrepot\brulerie.duckdb -f 01_de\sql\argent_torrefaction.sql
REM Attendu : argent 47, journal 47 ; refait en entier, l'argent ne double rien.
```

Puis, dans le notebook, l'`ATTACH`.

Étape 5 — Contrôler l'argent contre le journal · RAW

Le journal déclare, pour chaque lot, la seconde du premier craquement, la durée et la température de sortie ; l'argent les a tirées des points. Les deux doivent dire la même chose, lot par lot. Et une courbe sans trou compte une lecture toutes les 10 secondes, de la charge à la sortie : la durée divisée par 10, plus une.

5.1 Le témoin, dans une cellule neuve.

```
-- 1. chaque lot de l'argent, joint à sa ligne du journal
SELECT COUNT(*) AS lots,
       -- 2. un écart : une mesure qui diffère du journal, ou une lecture qui manque à la courbe
       COUNT(*) FILTER (WHERE a.craquement_s <> CAST(l.premier_craquement_s AS INTEGER)
                        OR a.duree_s <> CAST(l.duree_s AS INTEGER)
                        OR a.temp_sortie <> CAST(l.temp_sortie AS DECIMAL(5, 1))
                        OR a.lectures <> ____) AS ecarts
FROM entrepot.argent_lots AS a
JOIN entrepot.raw_lots AS l USING (lot_id);
-- Attendu : 47 lots, 0 écart.
```

Ce que le logiciel du torréfacteur déclare, ses sondes l'ont vu : 47 lots, aucun écart. Le jour où une sonde se tait à la sortie d'un lot, sa température de sortie manque, et le témoin le voit.

5.2 Lire le développement par profil.

```
SELECT profil, COUNT(*) AS lots, ROUND(AVG(developpement_pct), 1) AS developpement_moyen
FROM entrepot.argent_lots
GROUP BY profil
```


© 2026 iAlexMG. Tous droits réservés.

```
ORDER BY developpement_moyen;
-- Attendu : CLAIRE 3 lots à 12.7, MOYENNE 26 à 17.8, MOYENNE_FONCEE 18 à 23.6.
```

Plus le profil est foncé, plus la torréfaction se prolonge après le premier craquement : la mesure tirée des points dit ce que le métier attend. Boréal range en plus sa série dans une base de séries temporelles, pour la lire pendant qu'elle s'écrit ; à 3 718 lectures par mois, l'entrepôt suffit, et cette base reste à Boréal.

5.3 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 6 — Versionner · RAW

6.1 Valider le script, dans la première fenêtre.

```
git -C 01_de add sql/argent_torrefaction.sql
git -C 01_de commit -m "Cours 08 : les lots du torréfacteur dans l'argent, tirés de leurs courbes"
REM Attendu : 1 file changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Une série temporelle se qualifie avant de servir : sa clé, le type de son temps, son pas, ses sondes et son horloge, chacun par une requête.
- Une mesure se tire des points, et ce que le logiciel déclare se recoupe avec ce que ses sondes ont vu : 47 lots, aucun écart.
- L'argent ne prend que les lots dont la courbe est arrivée : le journal connaît le mois d'avance, les courbes arrivent jour après jour.

© 2026 iAlexMG. Tous droits réservés.